Système de Market Making par Apprentissage par Renforcement

Carnet d'ordres haute performance & agents de trading par RL

Bella Taleb El Alem Yani Haouchine Ihab Jaouad Anas Benbaki Mohamed Aziz Migaou Soufiane Sbai

ENSEIRB-MATMECA
PFA

7 mai 2026

Plan

Infrastructure & Déploiement

1. Contexte & objectifs
2. Architecture globale
3. Moteur C++ (matching, types d'ordres)
4. Pipeline Binance (données temps réel)

Stratégies & Résultats

5. Stratégies par règles
6. Agents RL (DQN, A2C, PPO)
7. Dreamer v3 (model-based)
8. SAC (market making off-policy)
9. Dashboard (monitoring + auth)
10. Bilan & perspectives

Le défi des marchés modernes

- 10^5 ordres/s
- Latence $\sim$ microseconde
- Données temps réel

Notre projet

Une plateforme **de bout en bout** pour simuler, entraîner et évaluer des stratégies sur données crypto.



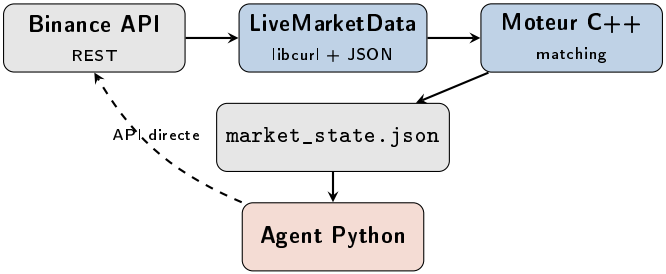
Données live : Binance API

Cahier des charges

Type	Exigence	Concrètement
Fonctionnel	Matching prix-temps, 5 types d'ordres	GTC, Market, IOC, FOK, GFD
Fonctionnel	Ingestion de données réelles	API Binance
Fonctionnel	Agents enfichables + visualisation	Dashboard web
Non-fct.	Performance haut débit	$\geq 10^5$ ops/s
Non-fct.	Latence opérationnelle	$\sim$ microseconde
Non-fct.	Reproductibilité	Seeds, checkpoints

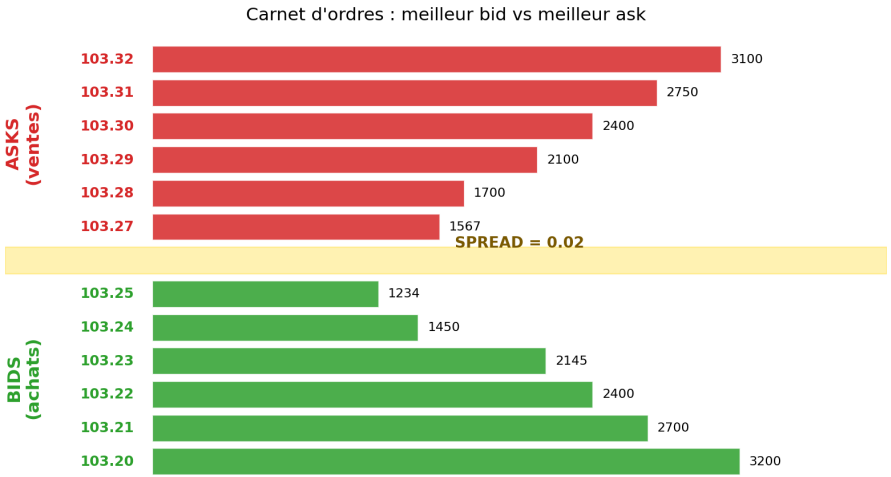
Hors périmètre : trading réel, conformité, FPGA.

Architecture logicielle



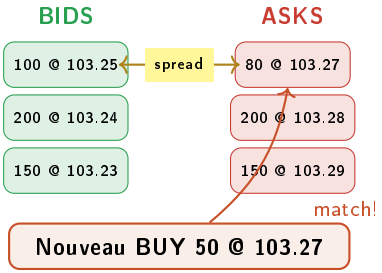
Chaîne simplifiée : l'Agent Python interagit directement avec l'API Binance, tandis que le moteur C++ traite le flux marché.

Carnet d'ordres



Bids = ordres d'achat (prix décroissants) | Asks = ordres de vente (prix croissants)

Matching prix-temps



Règles

1. Meilleur prix d'abord
2. À prix égal : FIFO
3. Exécutions partielles permises

Match dès que :

$$P_{\text{buy}} \geq P_{\text{ask}}$$

Structures de données

```
std::map<Price, OrderPointers, std::greater<Price» bids_;
std::map<Price, OrderPointers, std::less<Price» asks_;
std::unordered_map<OrderId, OrderEntry> orders_;
```

Op.	**Complexité**	**Débit**
Ajout	$O(\log n)$	$\approx$ 400 k/s
Annulation	$O(1)$	$\approx$ 2 M/s
Modification	$O(\log n)$	$\approx$ 270 k/s
Matching	$O(k \log n)$	$\approx$ 350 k/s

Cinq types d'ordres

GoodTillCancel
actif jusqu'à annulation

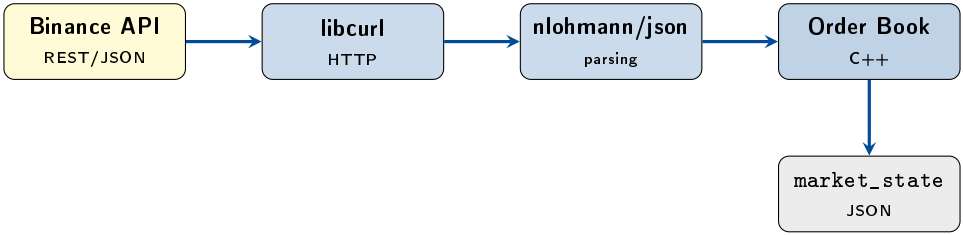
FillOrKill
tout ou rien

GoodForDay
expire à 15 :59

Tests fonctionnels

Suite `OrderBookTests` couvre les 5 types : insertion, annulation, modification, exécution partielle.

Pipeline d'ingestion temps réel



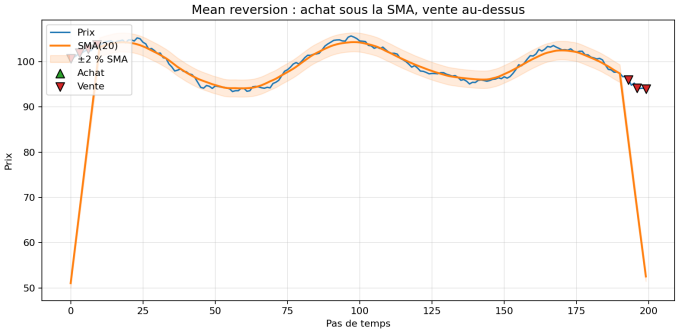
Lancement

```
./LiveMarketData SOLUSDT 1 20
```

Sorties live

- Bid / Ask / Mid / Spread
- Profondeur, SMA, PnL

Stratégie : Mean Reversion



Formules

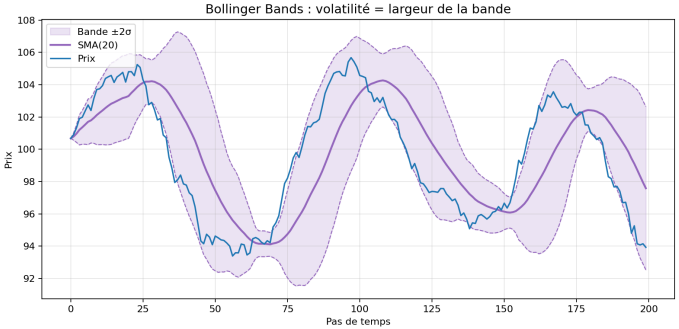
SMA

$$\mathrm{SMA}_n(t) = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i}$$

Signal

$$\begin{cases} \text{BUY} & P_t < \mathrm{SMA}_n - 2\% \\ \text{SELL} & P_t > \mathrm{SMA}_n + 2\% \end{cases}$$

Stratégie : Bollinger Bands



Formules

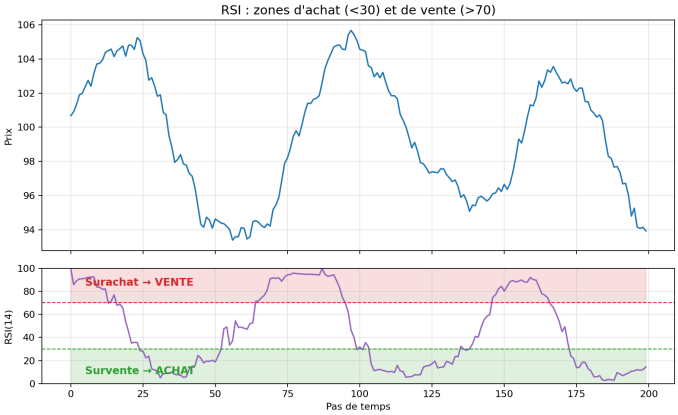
Écart-type

$$\sigma_n(t) = \sqrt{\frac{1}{n}\sum_{i=0}^{n-1}(P_{t-i} - \mathrm{SMA}_n)^2}$$

Signal

$$\begin{cases} \text{BUY} & P_t < \mathrm{SMA}_n - 2\sigma_n \\ \text{SELL} & P_t > \mathrm{SMA}_n + 2\sigma_n \end{cases}$$

Stratégie : RSI (Relative Strength Index)



Formules

RSI

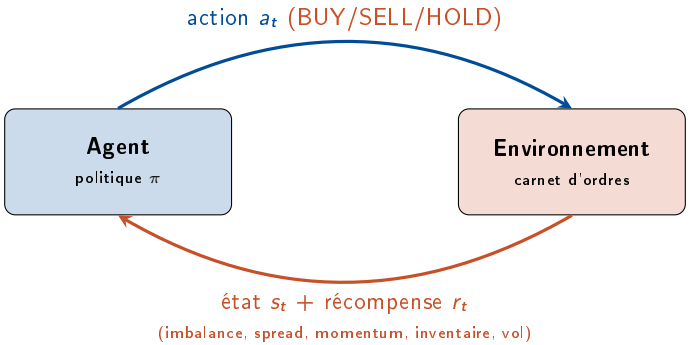
$$\mathrm{RSI}_n = 100 - \frac{100}{1 + RS}$$

où $RS = \frac{\text{gain}}{\text{perte}}$

Signal

$$\begin{cases} \text{BUY} & \mathrm{RSI} < 30 \\ \text{SELL} & \mathrm{RSI} > 70 \end{cases}$$

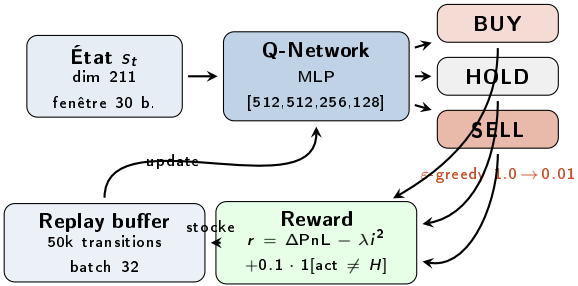
Apprentissage par renforcement



MDP du trading

L'agent observe l'état, choisit une action, reçoit une récompense (variation de PnL pénalisée par l'inventaire).

DQN – Fonctionnement dans notre contexte



Espace d'état (dim 211)

- 7 features $\times$ 30 bougies
- close, high, low, volume
- return, SMA5, SMA15
- inventaire normalisé $(+1)$
- volume bars (seuil 1000 SOL)

Équation de Bellman

$$Q(s,a) \leftarrow r + \gamma \max_{a'} Q_{\theta^-}(s',a')$$

$$\mathcal{L} = \left(Q_{\theta^-}(s',a') - Q_\theta(s,a)\right)^2$$

DQN – Implémentation et justifications

Architecture

Couches	[512,512,256,128]
Activation	ReLU
Sortie	dim 3 (B/H/S)

Profond :
Capacité pour dynamique non linéaire des crypto.

Stabilisation

Target network	sync 1000
Replay buffer	50k entrées
Batch	32
Loss	MSE
Grad clip	$[-5, 5]$

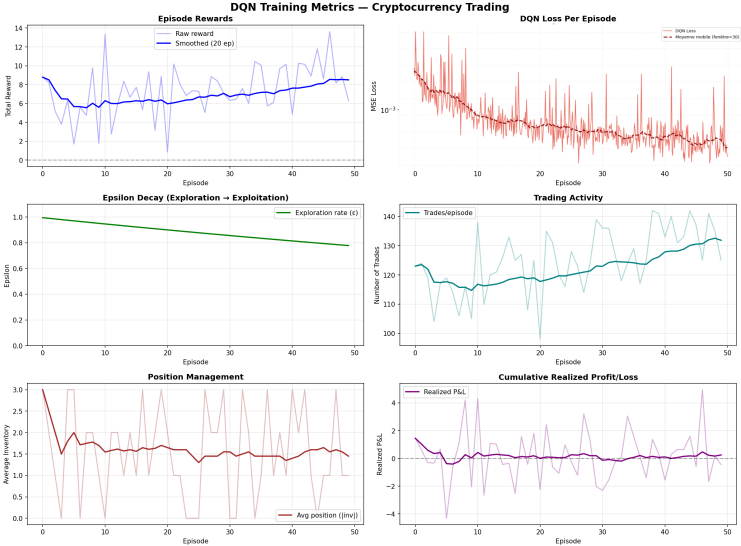
Target net : évite cible mouvante
Replay : brise corrélations

Hyperparamètres

γ	0.99
lr	10^{-3}
ε	$1.0 \rightarrow 0.01$
Decay	0.995/ep
λ inv.	0.01
Bonus action	$+0.1$
Épisodes	50

Reward shaping :
Pénalise *hold* passif

DQN – Apprentissage



A2C – Actor-Critic

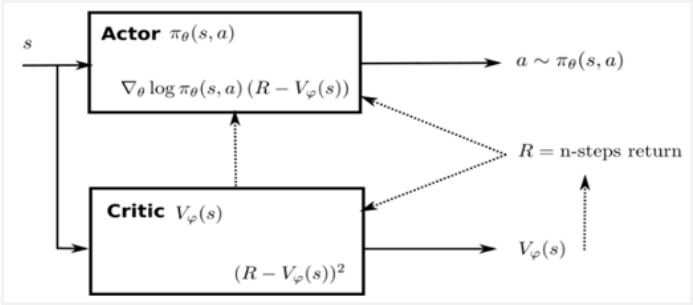
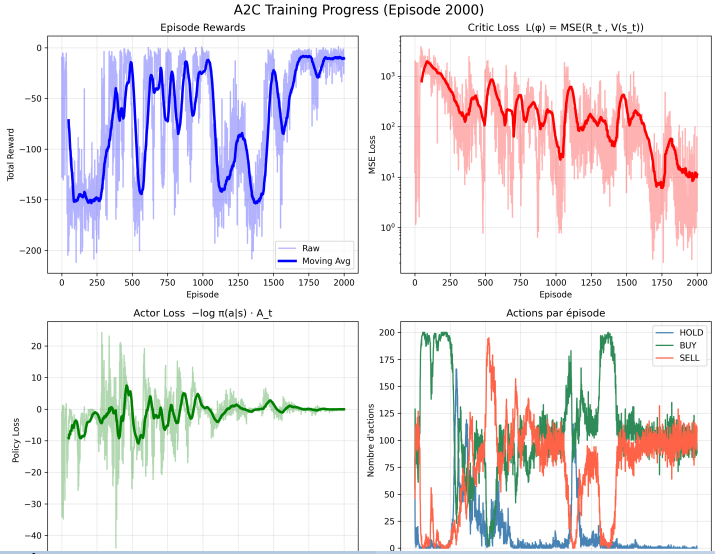


Figure – Principe général de l'architecture Actor-Critic utilisée pour A2C.

Avantage	$A_t = R_t - V(s_t)$
$\mathcal{L}_{\text{actor}}$	$-\log \pi(a_t \mid s_t)\, A_t$
$\mathcal{L}_{\text{critic}}$	$(R_t - V(s_t))^2$
Return	$R_t = r_t + \gamma R_{t+1}$

A2C – Actor-Critic



- La récompense s'améliore au fil de l'entraînement, mais reste négative.
- La perte du critique diminue, ce qui indique une meilleure estimation de la valeur.
- La politique se stabilise, mais la stratégie n'est pas encore rentable.

Hyperparamètres :

- lr_{actor} : 10^{-3}.
- lr_{critic} : 10^{-3}.
- $\gamma = 0.99$.
- Nombre d'épisodes d'entraînement : 2000.
- Coefficient de pénalité d'inventaire : 0.0001.

PPO – Politique, Loss & Reward

Politique continue (market making)

- Action $a = (a_{\text{ask}}, a_{\text{bid}}) \in [-1, 1]^2$
- Réseau acteur–critique **64–64** (séparé)

$$p_{\text{ask}} = p\left(1 + \tfrac{a_{\text{ask}}+1}{2}\, s_{\max}\right)$$

Reward

$$r_t = \underbrace{\Delta \text{cash}_t}_{\text{spread capturé}} - \underbrace{\lambda\, i_t^2\, p_t}_{\text{pénalité inventaire}}$$

Loss totale

$$\mathcal{L} = \mathcal{L}_\pi + 0{,}5\,\mathcal{L}_V - 0{,}01\,H(\pi)$$

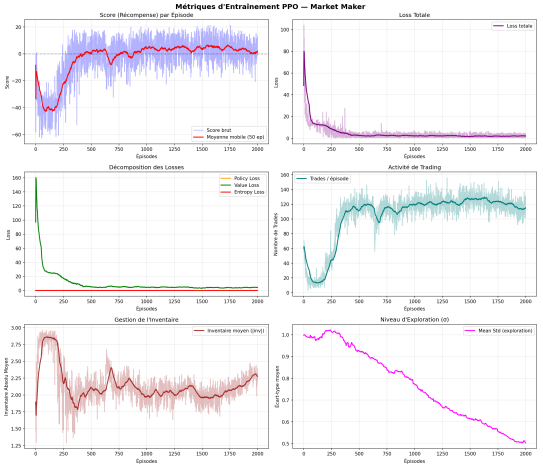
Policy loss (clipped surrogate) :

$$\mathcal{L}_\pi = -\mathbb{E}\Big[\min\big(r_t \hat{A}_t,\ \text{clip}(r_t, 1-\varepsilon, 1+\varepsilon)\hat{A}_t\big)\Big]$$

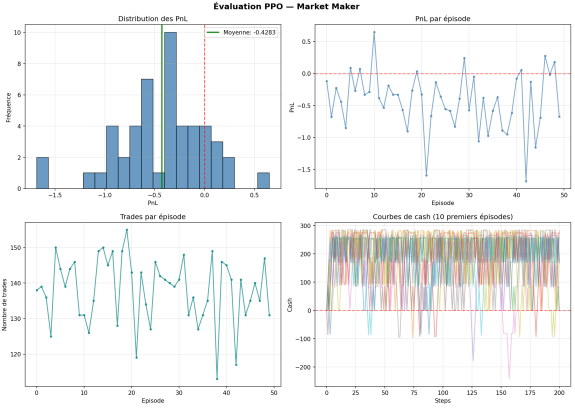
Variable	Définition
p	mid-price courant
$s_{\max} = 0{,}5\%$	spread maximum autorisé
$i_t \in [-10, 10]$	inventaire (SOL détenus)
$\lambda = 5 \times 10^{-4}$	coeff pénalité inventaire
$r_t = \pi_{\text{new}}/\pi_{\text{old}}$	ratio nouvelles/anciennes proba
$\hat{A}_t$	avantage GAE normalisé
$\varepsilon = 0{,}2$	borne de clipping PPO
$\gamma = 0{,}99$	facteur d'actualisation
$H(\pi)$	entropie de la politique

PPO – Entraînement & Évaluation

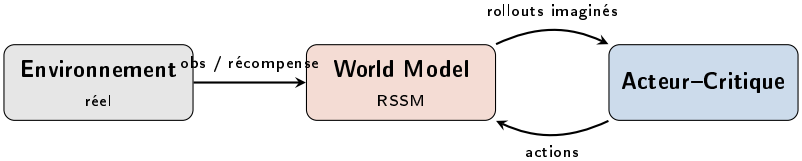
Entraînement (2 000 épisodes)



Évaluation hors-entraînement



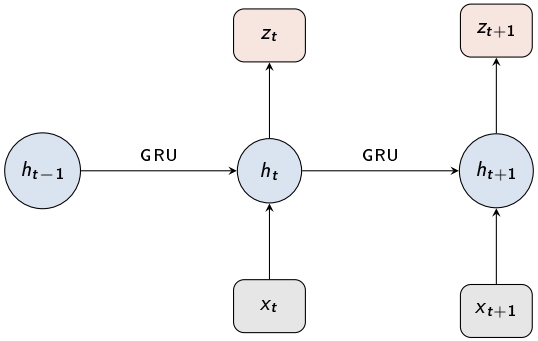
Dreamer v3 – Apprendre dans l'imagination



Idée clé

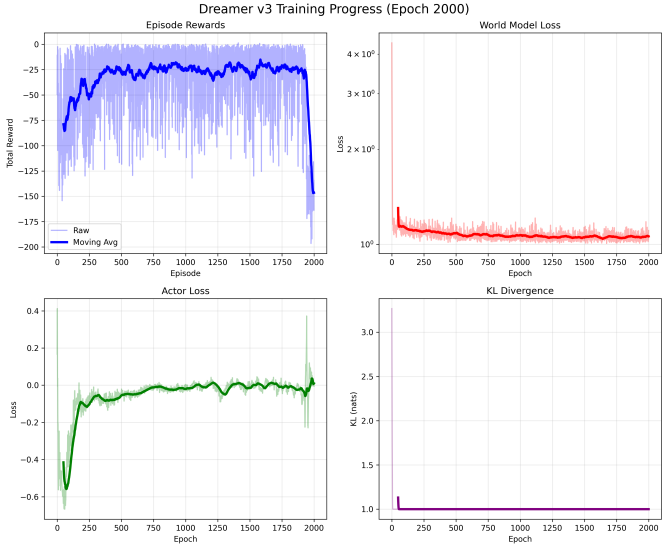
Apprendre la dynamique du marché dans un World Model, puis entraîner l'acteur sur des trajectoires imaginées — *sans* interagir avec l'environnement réel.

Dreamer v3 – World Model (RSSM)



Latents catégoriels 32×32 | GRU récurrent | symlog pour stabilité numérique

Dreamer v3 – Apprentissage



SAC – Market Making adaptatif

Différence vs DQN / A2C

- Action continue : $(b_s,\, a_s) \in [0,\, 0.5\%]^2$
- Pas de pari directionnel
- Capturer le spread des deux côtés

Reward Avellaneda-Stoikov

$$r = \underbrace{\Delta mid \times qty}_{\text{spread capturé}} - \underbrace{\lambda \frac{I^2}{\sigma}}_{\text{risque inventaire}} - \underbrace{\lambda_{adv}\, \delta_{adv}}_{\text{sélection adverse}}$$

$\Delta mid \times qty$	profit sur chaque ordre rempli
I	inventaire (position accumulée)
σ	volatilité du marché
λ	poids de la pénalité inventaire
δ_{adv}	perte due aux traders informés

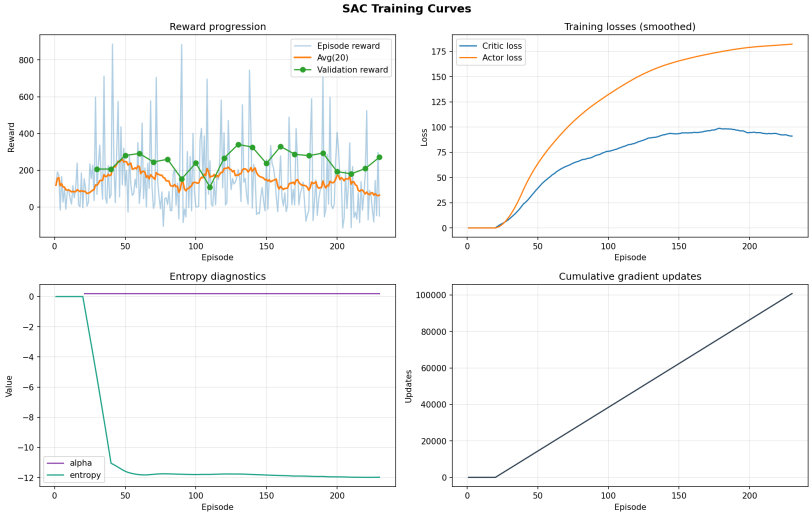
Architecture

Twin Q-net	anti-surestimation
$\alpha = 0.2$ fixe	entropie contrôlée
Buffer 500k	off-policy stable
$\tau = 0.005$	soft target update

Off-policy vs On-policy

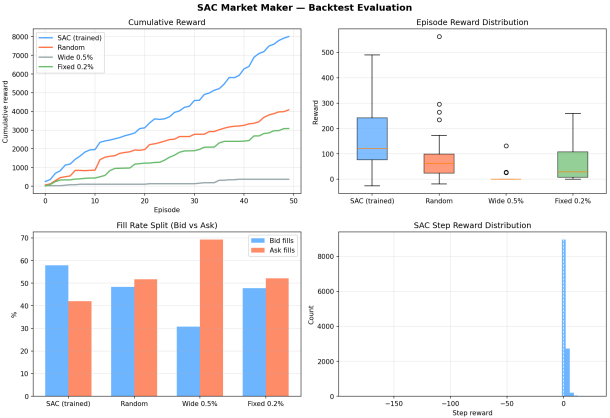
PPO/A2C	apprend de l'épisode en cours
SAC	réutilise 500k expériences passées

SAC – Courbes d'apprentissage



Meilleur checkpoint ep. 130 | Val reward 340.42 | Early stop ep. 230 (patience 100, 2500 max)

SAC – Évaluation (5 seeds × 50 épisodes, test set BTCUSDT)



Résultats clés

- Mean R $= 160.68 \pm 4.86$
- Sharpe $= 8.82 \pm 0.82$
- MaxDD $= 7.09 \pm 10\%$
- Fill rate $= 18.3\%$

Politique	Mean R	Sharpe
SAC	160.68	8.82
Random	84.92	6.37
Fixed 0.2%	58.06	4.62
Wide 0.5%	7.11	2.59

Dashboard – Interface temps réel

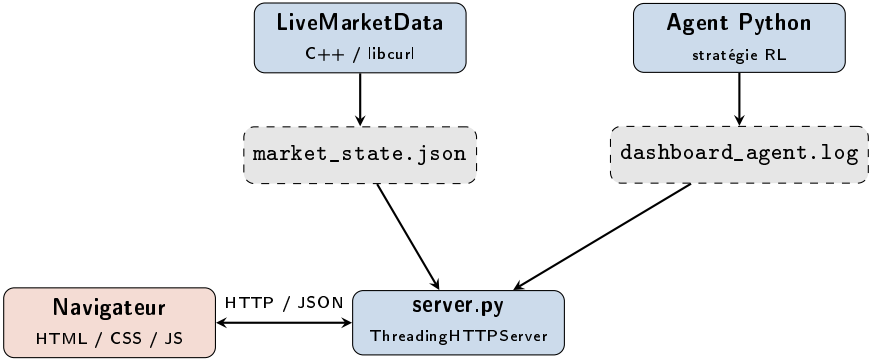
Composants UI

- Chandelier LightweightCharts (2 s)
- Carnet d'ordres live – 8 niveaux bid/ask
- Courbe PnL SVG – Δ équité
- Badge décision BUY / SELL / HOLD
- Barre inventaire – risque coloré
- Logs Agent / Market / Erreurs

Contrôles

- Sélection agent & paire à chaud
- Redémarrage auto si simulation active
- Authentification

Dashboard – Architecture découplée



Principe clé

Observateur pur – le dashboard ne bloque jamais l'agent. Zéro latence ajoutée au trading.

Bilan & perspectives

Réalisé

- Moteur C++ : $O(\log n)$ insert, $O(1)$ annulation, latence μs
- Ingestion temps réel Binance
- Dashboard authentifié, zéro latence ajoutée
- 3 stratégies par règles (MR, RSI, Bollinger)
- 4 agents RL : DQN, A2C, PPO, SAC

Perspectives

- Enrichir l'état avec le déséquilibre des flux d'ordres
- Exploiter plusieurs niveaux de profondeur du carnet
- Récompense ajustée au risque (ratio de Sharpe)

Conclusion

Plateforme **de bout en bout** : moteur, données, agents RL et le dashboard.

Merci de votre attention — Questions ?